

Project Report: Karachi AQI Prediction Dashboard

1. Overview:

This project forecasts Karachi's Air Quality Index (AQI) using real-time pollutant and weather data via a fully automated ML pipeline. It handles live data fetching, preprocessing, feature engineering, cloud storage (Hopsworks), and automated retraining (GitHub Actions CI/CD), visualized through a Streamlit dashboard and send alerts through discord.

AQI is computed per U.S. EPA standards (May 2016) using truncation, breakpoint mapping, and linear interpolation for PM_{2.5}, PM₁₀, CO, NO₂, SO₂, and O₃. Open-Meteo APIs (Air Quality, Weather Forecast, and Historical Archive) were chosen for reliability, pollutant coverage, and hourly forecasts.

2. Data Collection & Preprocessing:

Scripts: `fetch_data.py`, `process_data.py`, `merge_features.py`, `backfill_data.py`.

- Merged pollutant and weather datasets (Jan 2024–Nov 2025) into a unified hourly DataFrame with standardized units ($\mu\text{g}/\text{m}^3$).
 - Outliers capped (1st–99th percentile) for PM_{2.5}, PM₁₀, CO; NO₂ retained for genuine peaks.
 - Correlation ($r=0.53$) showed PM_{2.5} & PM₁₀ as key AQI drivers.
 - Observed winter buildup, dusty summer peaks, and spring ozone rise.
- Clean dataset ready for AQI computation.

3. AQI Computation & Feature Engineering:

AQI (`aqi_utils.py`): Converted pollutant concentrations to EPA units and calculated sub-indexes; final AQI = $\max(\text{sub-indexes})$.

Features (`process_features.py`):

- Time: hour, weekday, month, cyclic encodings.
- Trends: 24h rolling mean, 1h lag.
- Derived: PM_{2.5}/PM₁₀, temp-humidity ratio, wind dispersion, high-pollution flag.
- Missing values interpolated.

→ Final dataset: 23 engineered features.

4. Feature Refinement:

Analyzed 15,744 records, 40 variables.

- Removed redundant/highly correlated (>0.9) features.
 - Retained PM_{2.5}, PM₁₀, SO₂, O₃ as core predictors.
 - Found AQI highest in Dec–Jan mornings due to temperature inversion.
- Exported as `final_selected_features.csv`.

5. Cloud Integration (Hopsworks):

Scripts: `feature_store.py`, `run_feature_pipeline.py`.

- Secure upload via API key to versioned Feature Group `aqi_features_v1`.
- Daily updates, no duplication (timestamp-based).
- Local CSV saving disabled for CI/CD.

6. Model Training & Evaluation:

Script: `train_model.py`.

- Data fetched from Hopsworks; dropped leakage-prone features; added $\pm 5\%$ Gaussian noise.
- Time-based 80/20 split. Models: Ridge, Random Forest, XGBoost, Gradient Boost .

Model	RMSE	MAE	R ²
XGBoost	6.70	1.28	0.987
Gradient Boosting	7.91	0.58	0.982
Random Forest	8.17	0.34	0.981
Ridge Regression	34.70	16.29	0.655

7. CI/CD Automation:

Implemented via GitHub Actions:

Workflow	File	Time (PKT)	Function
Feature Pipeline	<code>feature_pipeline.yml</code>	8:10 AM	Fetch, process, upload to Hopsworks
Training Pipeline	<code>training_pipeline.yml</code>	8:30 AM	Retrain ML model

→ Ensures daily updates and retraining automatically.

8. Streamlit Dashboard (Locally for now):

Files: `app.py`.

- Loads latest model; predicts AQI for current + next 3 days.
 - Displays 3-day bar chart, hourly line chart, and AQI category cards (EPA breakpoints).
- Real-time, interactive visualization of Karachi's air quality.

9. Challenges & Fixes

Issue	Resolution
Future-day overlap	Adjusted UTC window
Duplicate entries	Enforced datetime_str primary key
Workflow timing mismatch	Updated cron jobs
API/dependency errors	Added confluent-kafka, version-pinned hopsworks[python]

10. Conclusion

The Karachi AQI Prediction Dashboard delivers a production-grade, automated MLOps pipeline that:

- Fetches & cleans live data per EPA AQI standards.
- Generates engineered features and uploads them to Hopsworks.
- Trains and retrains models via CI/CD.
- Visualizes predictions through Streamlit.
- Serves real-time predictions via a high-performance **FastAPI** REST interface.
- Triggers instant Discord alerts to notify stakeholders whenever air quality reaches **hazardous levels**.